Khulna University of Engineering & Technology
Department of Computer Science and Engineering

Final Project Report

FixFlow

Electronic Device Repair Management System

A role-based web application for repair request, diagnosis, invoicing, payment, fulfillment, and reporting using Laravel.

Course Information

Course	CSE 3100 — Web Programming Laboratory
Lab Group	B2
Submission Date	19 July 2026
Technology Stack	Laravel 13 · PHP 8.3+ · PostgreSQL · Stripe · Reverb

Project Highlights

- Multi-role portals for Customer, Technician, and Admin
- Repair lifecycle with diagnosis, quote approve/decline, and status timeline
- Stripe Checkout payments, invoices, fulfillment, and warranties
- Realtime per-repair chat (Laravel Reverb) and admin reports

SUBMITTED TO

Mr. Kazi Saeed Alam
Assistant Professor
Department of CSE, KUET

Ehsanul Karim Talha
Lecturer
Department of CSE, KUET

SUBMITTED BY

Mynul Hassan Mehadi
Roll: 2207105
Lab Group: B2
Department of CSE, KUET

Abstract

This project presents FixFlow, a web-based electronic device repair management system. The main goal is to digitize the full repair lifecycle so customers, technicians, and administrators share one platform instead of paper tickets, phone calls, and disconnected billing.

On the customer side, a user can submit a repair request, track status on a timeline, chat with the assigned technician, pay an invoice online (Stripe) or via admin-recorded payment, choose pickup or home delivery after payment, and view warranties. Technicians update diagnosis and status on assigned jobs. Admins assign technicians, approve applicants, review and send invoices, confirm fulfillment, issue warranties, and view reports.

The application is built with Laravel 13 (MVC), Blade + Tailwind CSS, PostgreSQL, Laravel Reverb for live chat, and Stripe Checkout for payments. Schema is managed with migrations and seeders; authorization uses session auth with role and approved-technician middleware. Core flows were verified with feature tests and seeded demo accounts.

From a software-engineering view, FixFlow emphasizes clear domain boundaries: repair requests own status and diagnosis; invoices and warranties stay 1:1 with a completed repair; messages belong to a repair thread; technician applications gate privileged access. Business rules for assignment, payment, fulfillment, and warranty issue live in controllers and model helpers so the UI cannot bypass workflow constraints.

The external integration used in production-like demos is Stripe Checkout (test mode): customers pay unpaid invoices through a hosted session, and webhooks or success callbacks mark invoices paid. Realtime chat uses self-hosted Laravel Reverb rather than a hosted Pusher account. Email for password reset can run through local Mailpit during development or any SMTP provider in deployment.

8 Core tables	**3** User roles	**12** Controllers	**10** Design figures
1:1 Invoice / repair	**Reverb** Live chat	**Stripe** Online pay	**MVC** Architecture

Keywords: Laravel, MVC, Eloquent, Blade, PostgreSQL, ER model, DFD, UML, Stripe, WebSockets, role-based access, repair workflow, invoicing, fulfillment.

Project Contribution Summary

The delivered system demonstrates an end-to-end academic web application: requirement analysis, conceptual and physical data modeling, MVC implementation, role-based security, payment and realtime integrations, and verification with seeded demo data and automated feature tests. The report documents architecture, DFDs, ER/UML diagrams, module maps, and deployment steps so the design can be reproduced and extended.

Deliverables

- Working Laravel web app with three roles.
- Migrations, seeders, and demo accounts.
- Design report with ER, DFD, and UML figures.
- Stripe test-mode payment path and webhooks.
- Reverb-backed live chat per repair.

Evaluation Focus

- Correct repair lifecycle transitions.
- Invoice draft → sent → paid integrity.
- Role and technician-approval gates.
- Fulfillment only after payment.
- Readable admin reports and audits.

Table of Contents

List of Figures

List of Tables

Document conventions: demo password is password for all seeded accounts; Stripe runs in test mode; repository at github.com/mehadi105/FixFLow.

1. Introduction and Objectives

1.1 Problem Statement

Traditional repair shops often rely on paper tickets, phone calls, and manual billing. Customers lack real-time visibility into repair progress; technicians need clear assigned jobs and status transitions; admins need centralized control over users, invoices, and revenue. Payment and device return are frequently disconnected from the repair ticket, and communication usually happens off-platform.

These gaps create repeated status inquiries, delayed payments, lost paper records, and unclear responsibility when a device is ready for pickup or delivery. Without a shared system of record, shops also struggle to produce simple operational reports such as open jobs, unpaid invoices, or technician workload.

1.2 Proposed Solution

FixFlow covers the repair cycle in software: submit a request, assign an approved technician, update diagnosis and status, auto-create a draft invoice on completion, admin send/pay (Stripe or manual), choose pickup or delivery, confirm fulfillment, issue warranty, chat per repair thread, and show admin reports. Authorization and workflow rules are enforced in Laravel middleware and model helpers, not only in the UI.

Each actor sees a role-specific dashboard. Customers follow a timeline; technicians work only on assigned jobs; admins oversee applications, assignments, invoices, fulfillment, and warranties. Stripe Checkout is the online payment path; admin-recorded payments cover walk-in or cash cases. After payment, fulfillment and warranty steps close the loop.

Primary Objectives

- Secure multi-role authentication and dashboards.
- Full repair lifecycle with timeline tracking.
- Draft-to-paid invoicing with Stripe or manual pay.
- Pickup/delivery fulfillment after payment.
- Live messaging and unread notifications.
- Admin reports with joins and aggregates.

Learning / Design Objectives

- Model the system with ER, DFD, and UML.
- Apply MVC, migrations, Eloquent relations.
- Use middleware for roles and rate limits.
- Integrate payments and WebSocket chat.
- Demonstrate transactions and integrity rules.

1.3 Scope

Included	Outside Current Scope
Customer/technician/admin auth; repair CRUD; assignment; status/diagnosis; draft invoices; Stripe/manual pay; fulfillment; warranties; chat; reports; technician applications; password reset.	Native mobile apps, spare-parts inventory, GPS courier tracking, multi-branch SaaS billing, and production multi-tenant deployment.

1.4 Stakeholders and Success Criteria

Primary stakeholders are customers (device owners), technicians (service providers), and admins (shop operators). Secondary stakeholders include course instructors evaluating design quality and future maintainers extending the codebase.

- A customer can create a request, see status updates, chat, pay, and choose fulfillment.
- A technician can update only assigned jobs and communicate inside the repair thread.
- An admin can approve technicians, assign work, manage invoices, and view reports.
- Data integrity holds: one invoice and at most one warranty per repair; paid before fulfillment.
- Success criteria: reproducible demo — migrate + seed, sign in with role accounts, walk a repair through quote/payment/fulfillment, and show matching design artifacts.

2. Requirements and Actors

2.1 Customer Requirements

1. Register and sign in with role-based dashboard access.
2. Create a repair request with device details and optional image.
3. Track repair status on a timeline.
4. Chat with participants on a repair thread.
5. Pay unpaid invoices via Stripe Checkout.
6. Choose pickup or home delivery after payment.
7. View issued warranties.
8. Reset password via emailed link.

2.2 Technician Requirements

1. Apply to join as a technician and await admin approval.
2. View assigned repair jobs on the technician dashboard.
3. Update status (diagnosing, repairing, completed).
4. Save diagnosis notes for a repair.
5. Chat with the customer on assigned repairs.

2.3 Administrator Requirements

1. Manage users and roles.
2. Approve or reject technician applications.
3. Assign approved technicians to pending repairs.
4. Update any repair; review/send/delete draft invoices.
5. Mark invoices paid; complete fulfillment.
6. Issue warranties; view analytics reports.

2.4 Non-Functional Requirements

Quality	Requirement and Implementation
Integrity	Migrations, FKs, unique constraints; one invoice/warranty per repair.
Security	CSRF, hashed passwords, role middleware, rate limiting, Stripe webhook signature.
Reliability	Stripe success URL + webhook dual path; session regenerate on login.
Usability	Responsive Blade UI, status timeline, inbox-style messaging.
Performance	Pagination, eager loading, AJAX badges, named throttle limiters.
Maintainability	MVC separation, Invoice/Stripe services, seeders, feature tests.

2.5 Core Business Rules

Role-scoped access | Approved technician only | Draft invoice on complete | Customer cannot see drafts

Pay then choose fulfillment | Chat participants only | 1:1 invoice per repair | 1:1 warranty per repair

3. UML Use-Case Diagram

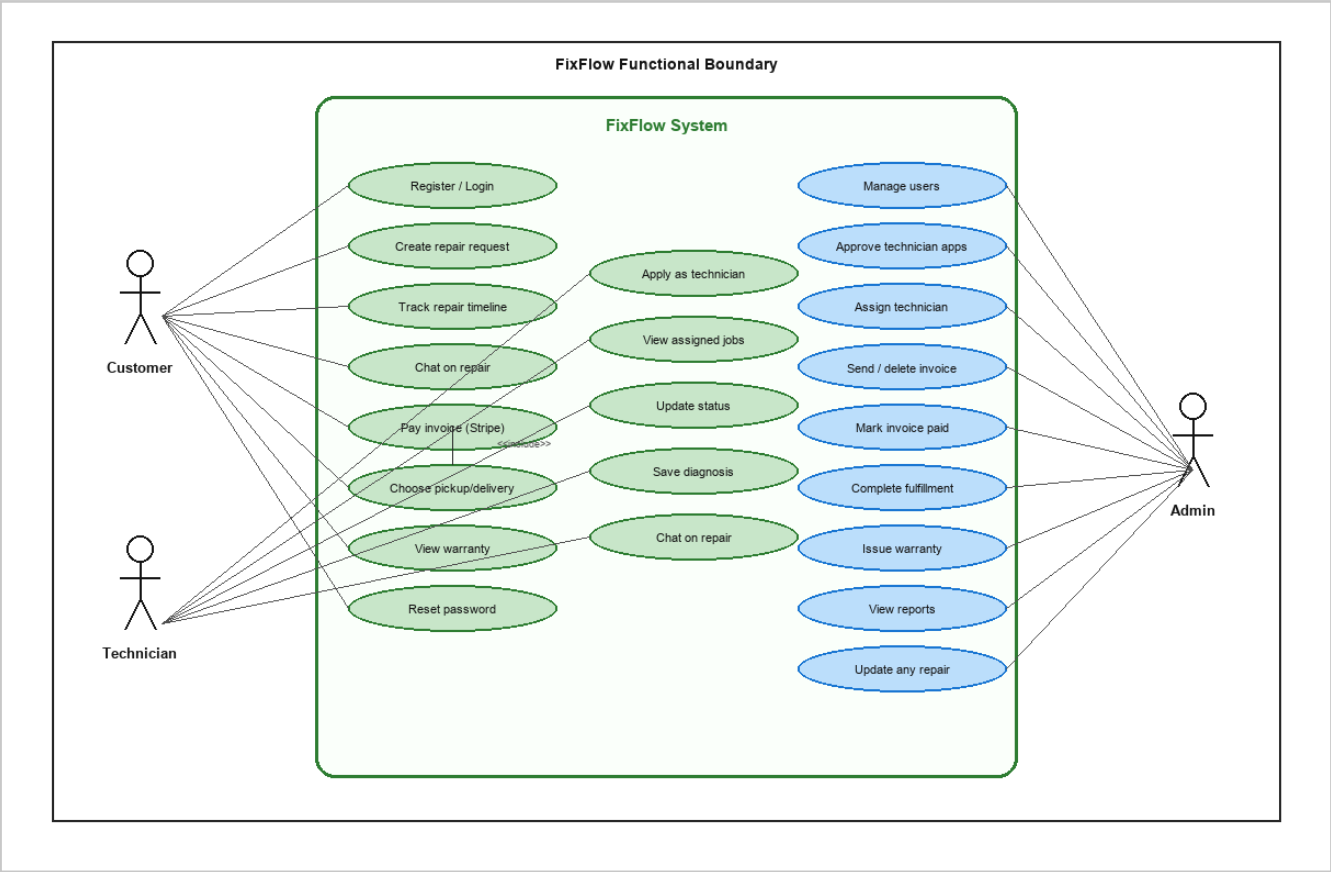

Figure 3.1 - Role-specific use cases inside the FixFlow system boundary.

3.1 Use-Case Summary

Actor	Primary Use Cases	System Effect
Customer	Register/login, create repair, track, chat, pay, pickup/delivery, warranty	Writes repairs, messages, payments, fulfillment choice
Technician	Apply, view jobs, update status, diagnosis, chat	Updates repairs; writes messages; application row
Admin	Users, approve apps, assign, invoices, fulfillment, warranties, reports	Writes users/invoices/warranties; reads analytics

4. System Architecture

FixFlow follows a classic three-layer web architecture: a Blade/Tailwind presentation layer, a Laravel application layer (controllers, middleware, services), and a PostgreSQL data layer with file storage for device images.

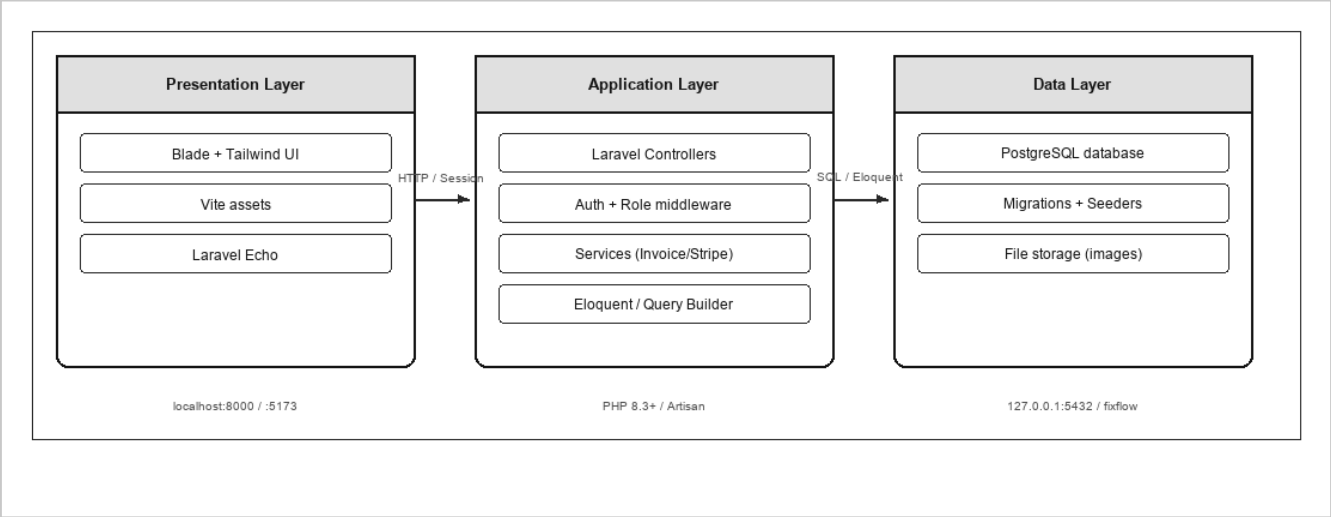

Figure 4.1 - Three-layer architecture and local ports (black-and-white).

Layer	Key Technologies	Responsibility
Frontend	Blade, Tailwind CSS 4, Vite 8, Echo/pusher-js	Role-specific screens, forms, chat UI, charts/badges.
Backend	Laravel 13, PHP 8.3+, middleware, services	Auth, authorization, validation, orchestration, Stripe.
Database	PostgreSQL, Eloquent, Query Builder, migrations	Persistence, constraints, aggregates, seed data.
Realtime / Pay	Laravel Reverb, Stripe Checkout	Live chat events; card payments in test mode.

4.1 Local Runtime Ports

- Web app: php artisan serve → http://127.0.0.1:8000
- Vite HMR (dev): npm run dev → http://localhost:5173
- Reverb WebSockets: php artisan reverb:start → port 8080
- PostgreSQL: 127.0.0.1:5432, database fixflow

> PostgreSQL is the default database. Reverb and Mailpit are optional local processes for live chat and password-reset email. Stripe runs in test mode with webhook/success confirmation.

5. Data Flow Diagrams - Level 0 and Level 1

5.1 Level 0 - Context

Customer, Technician, Admin, Stripe, and Email exchange data with FixFlow; role access and chat participation are enforced inside the app.

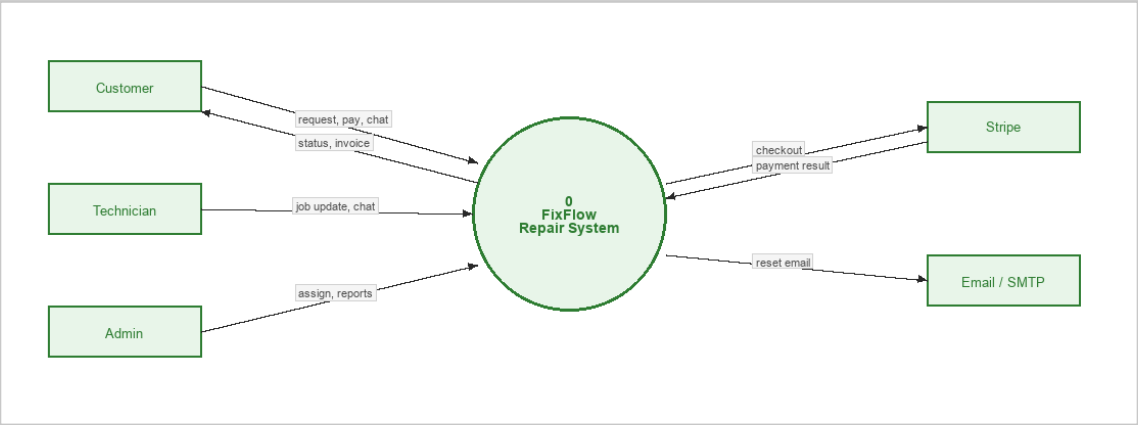

Figure 5.1 - Context diagram (Level 0).

5.2 Level 1 - Major Processes

Auth, Repairs, Tech Work, Invoice/Pay, Fulfillment, Messaging, Reports, and Tech Apps; stores D1-D5 map to core tables.

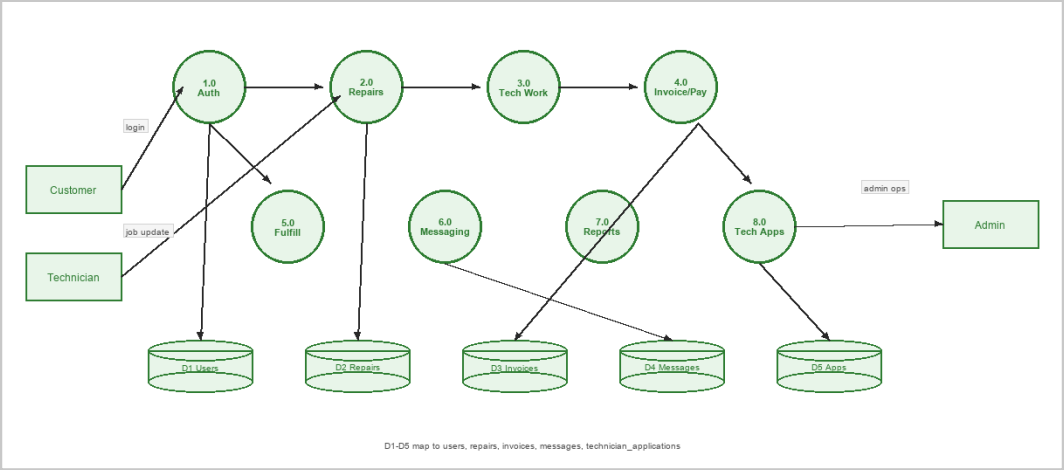

Figure 5.2 - Level-1 processes and data stores (D1-D5 map to core tables).

5.3 Process-to-Store Map

Process	Writes / Reads	Store
1.0 Auth	login, session	D1
2.0-3.0 Repairs / Tech	requests, status, diagnosis	D2
4.0-5.0 Invoice / Fulfill	invoices, payments, delivery	D3
6.0 Messaging	chat messages, read state	D4
7.0-8.0 Reports / Apps	analytics, applications	D5

6. Entity-Relationship Diagram

ER diagram from the live PostgreSQL schema (DBML / dbdiagram.io). Yellow = PK; teal = FK. Quote columns on REPAIR_REQUESTS support post-diagnosis approve/decline.

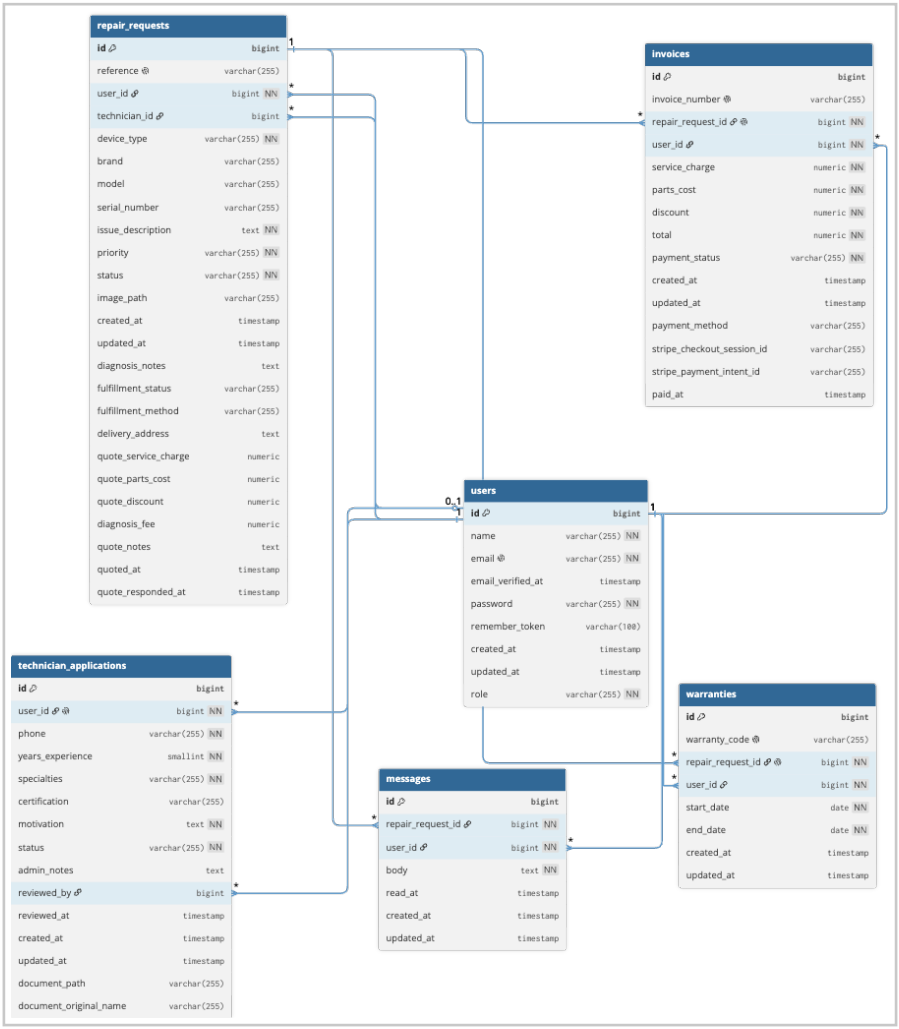

Figure 6.1 - FixFlow ER diagram (PostgreSQL domain schema).

6.1 Entity Overview

Entity	Role in Domain	Key Relationship
USERS	Identity and role	1:N repairs as customer or technician
REPAIR_REQUESTS	Job ticket / lifecycle	N:1 customer; N:1 technician (nullable)
INVOICES	Billing for a repair	1:1 with repair_request
WARRANTIES	Post-repair guarantee	1:1 with repair_request
MESSAGES	Per-repair chat	N:1 repair; N:1 sender user
TECHNICIAN_APPLICATIONS	Onboarding gate	1:1 with applicant user

Integrity: unique email and repair reference; unique repair_request_id on invoices and warranties (1:1); message threads indexed by (repair_request_id, created_at); user FKs cascade or null per migration policy.

7. Physical Schema and Relational Design

Schema is versioned with Laravel migrations. Key decisions: unique email and repair reference; unique repair_request_id on invoices and warranties; composite index on messages (repair_request_id, created_at); cascade / nullOnDelete on user FKs.

7.1 Core Tables

Table	Key Columns	Constraints
users	id, name, email, role, password	email unique; role default customer
repair_requests	reference, user_id, technician_id, status	FKs to users; reference unique
invoices	repair_request_id, totals, payment_status	repair_request_id unique
warranties	repair_request_id, warranty_code, dates	repair_request_id unique
messages	repair_request_id, user_id, body, read_at	index on RR + created_at
technician_applications	user_id, phone, status, reviewed_by	user_id unique
password_reset_tokens	email, token, created_at	email PK
sessions	id, user_id, payload, last_activity	session driver storage

7.2 Status Domains

- Repair: pending, assigned, diagnosing, repairing, completed
- Invoice: draft, unpaid, paid
- Fulfillment: awaiting_invoice through fulfilled
- Application: pending, approved, rejected

7.3 Design Notes

- One invoice and one warranty per repair (unique FK).
- Chat indexed by repair_request_id + created_at.
- Technician applications bound 1:1 to user.
- Cascade / nullOnDelete on user foreign keys.

7.4 Cardinality Summary

Relationship	Cardinality	Notes
User (customer) - RepairRequest	1 : N	user_id FK cascade
User (technician) - RepairRequest	1 : N	technician_id nullable
RepairRequest - Invoice	1 : 1	unique repair_request_id
RepairRequest - Warranty	1 : 1	unique repair_request_id
RepairRequest - Message	1 : N	chat thread per repair
User - TechnicianApplication	1 : 1	unique user_id

8. Module Design and Implementation

8.1 Authentication and Authorization

Session login/register, PasswordResetController, EnsureUserHasRole, EnsureApprovedTechnician, and named rate limiters.

8.2 Repair Requests

Create with optional image; admin assignment; status/diagnosis updates; show page with timeline, invoice/warranty cards, and chat link.

8.3 Technician Applications

Public apply form creates technician user + pending application; admin approve/reject with notes.

8.4 Invoicing and Payments

InvoiceService creates drafts on completion; admin send/edit/delete; StripePaymentService + webhook/success mark paid; manual mark-paid supported.

8.5 Fulfillment

After payment, customer chooses pickup or delivery; admin confirms handover to fulfilled.

8.6 Messaging and Notifications

Per-repair inbox; MessageSent on private channels; Echo + polling fallback; unread badges and navbar notification feed.

8.7 Warranties and Reports

Admin-issued warranties; Eloquent aggregates plus DB::table()->leftJoin() technician report.

8.8 Controllers

Controller	Responsibility
DashboardController	Role dashboards and /dashboard redirect
RepairRequestController	Repairs, assign, status, diagnosis, fulfillment
InvoiceController	Draft/send/update/delete/mark-paid
InvoicePaymentController	Stripe checkout, success/cancel, webhook
MessageController	Inbox + JSON chat APIs
WarrantyController	List/issue warranties
ReportController	Admin analytics + joins
UserController	User list and role updates
TechnicianApplicationController	Apply/status/approve/reject
PasswordResetController	Forgot/reset password
NotificationController	Navbar notification JSON
PreferenceController	Table density cookie

9. Security, Validation, and Error Handling

9.1 Authentication and Session Security

- CSRF tokens on state-changing forms; AJAX sends X-CSRF-TOKEN.
- Session regenerate on login/register; invalidate + regenerateToken on logout.
- Passwords hashed via Laravel hashed cast / Hash::make.
- HttpOnly session cookie; SameSite lax; Secure optional in production.

9.2 Authorization and Access Control

- Role middleware prevents privilege escalation across portals.
- Approved-technician middleware blocks unapproved tech dashboards.
- Chat channel authorization via RepairRequest::hasChatParticipant.
- Customers cannot view draft invoices until admin sends them.

9.3 Rate Limiting and External Integrations

- Rate limiting on login, registration, password reset, AJAX, and message send.
- Stripe webhook verifies signature; CSRF excluded only for webhook path.

9.4 Validation and Error Handling

Area	Validation / Control	On Failure
Forms	Blade CSRF + server-side rules in controllers	422/redirect with errors
Auth	Credential check + throttle limiters	401/429 with safe message
Roles	EnsureUserHasRole / EnsureApprovedTechnician	403 abort
Payments	Stripe signature + session_id check	Reject webhook; no state change
Chat	Participant check + repair ownership	403 on private channel
Invoices	Draft visibility + payable state guards	403/404 as appropriate

Errors are surfaced through Laravel validation responses, HTTP abort codes, and flash messages in Blade views so users receive clear feedback without exposing internals.

10. Activity and Sequence Diagrams

10.1 Repair Lifecycle Activity

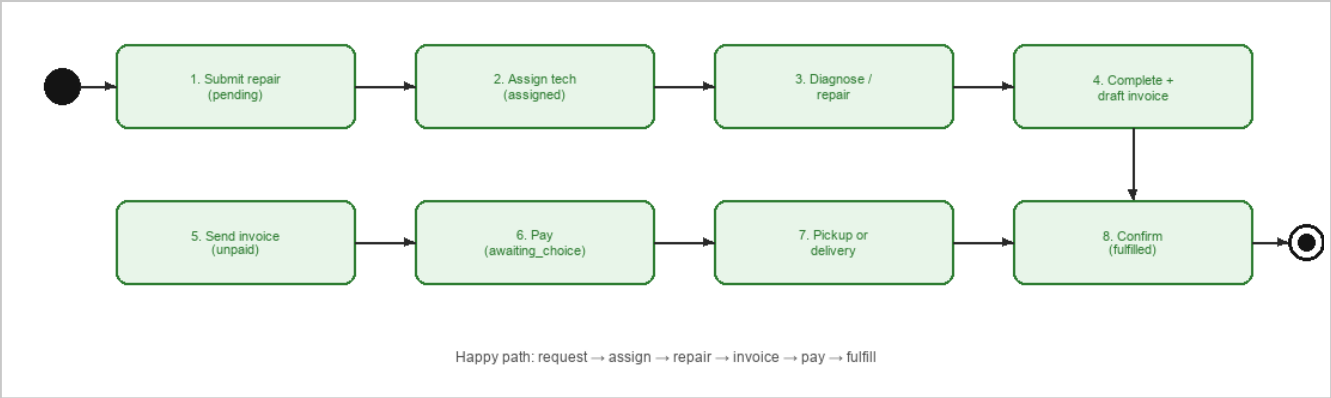

Figure 10.1 - Happy-path activity from request creation to fulfillment.

10.2 Sequence: Live Chat

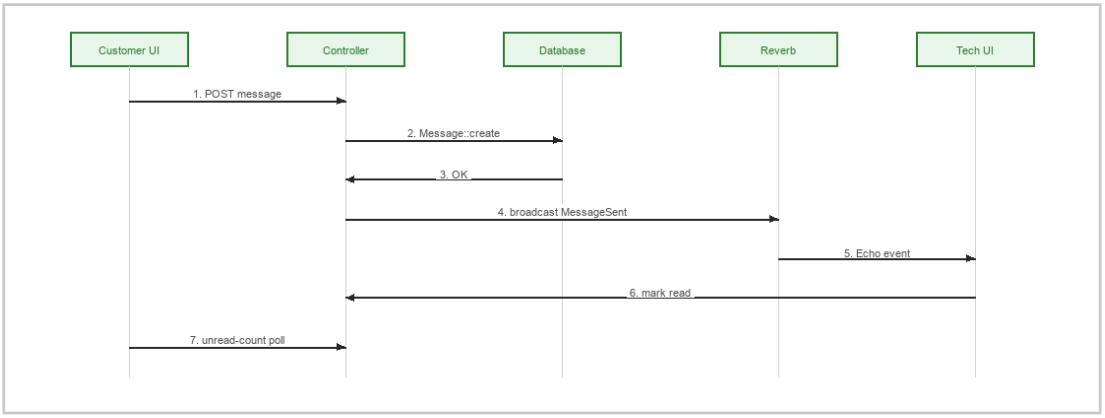

Figure 10.2 - Chat message broadcast and unread polling.

10.3 Sequence: Stripe Payment

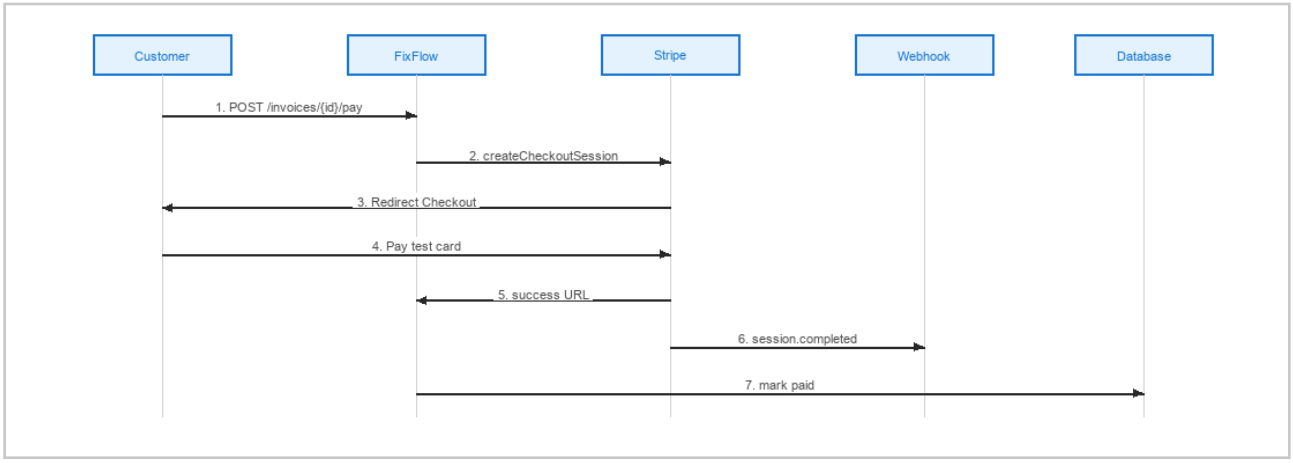

Figure 10.3 - Stripe Checkout (success URL + signed webhook both mark paid).

11. Class Diagram and Domain Models

Domain classes mirror Eloquent models under app/Models, including attributes, relationship methods, and key helpers used by controllers and policies.

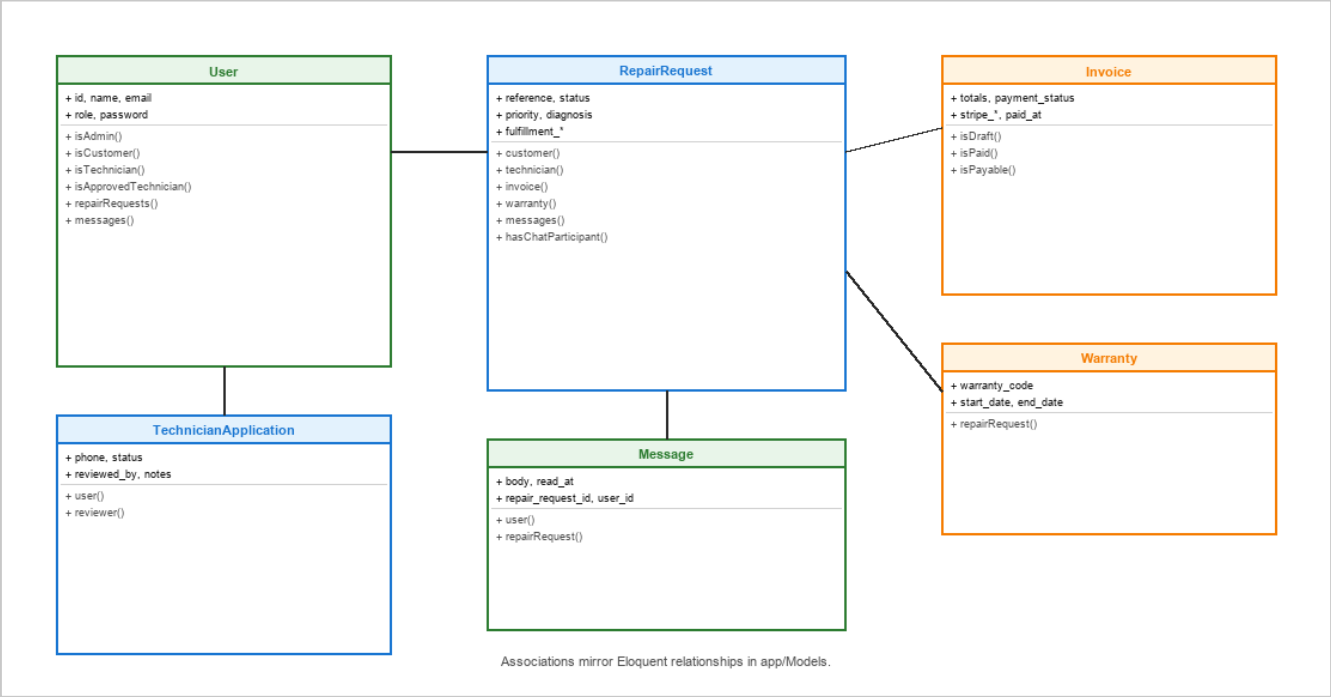

Figure 11.1 - UML class diagram of core domain models.

11.1 Key Helpers

- User::isApprovedTechnician() gates technician dashboard access.
- RepairRequest::hasChatParticipant() authorizes private chat channels.
- RepairRequest::canChooseFulfillment() enables pickup/delivery after pay.
- Invoice::isDraft / isPayable control admin send and customer pay actions.

12. Testing and Verification

Feature tests under tests/Feature cover response headers, login rate limiting, joined report data, preference cookies, and draft invoice deletion. Manual showcase flows cover chat, Stripe test payments, and fulfillment.

Test Focus	Approach
Smoke	GET / returns 200
Trace headers	X-Request-ID / X-FixFlow-App asserted
Rate limit	11th failed login returns 429
Reports join	Admin sees technician via leftJoin query
Cookie	POST preferences queues ff_table_density
DELETE	Draft invoice deleted; unpaid invoice protected
Manual E2E	Seeded demos RR-DEMO-PAY / RR-DEMO-PAY-2

13. Deployment and Demonstration

13.1 Local Setup

```
composer install && npm install
cp .env.example .env && php artisan key:generate
# Configure DB_* for PostgreSQL in .env, then:
php artisan migrate --seed
php artisan storage:link
npm run build
php artisan serve
php artisan reverb:start
mailpit
```

13.2 Showcase Script

- Admin: approve applicant; assign pending repair.
- Technician: update status/diagnosis; reply in Messages.
- Customer: open Messages; view progress threads.
- Admin: send RR-DEMO-PAY draft; customer pays RR-DEMO-PAY-2 (card 4242...).
- Customer: choose pickup/delivery; admin mark fulfilled; show Reports.

14. Conclusion and References

FixFlow demonstrates a complete academic web system for electronic device repair management. It integrates authentication, role-based access, repair workflow, Stripe invoicing, fulfillment, warranties, live chat, and reporting in a coherent Laravel MVC application. The design artifacts in this report (DFD, ERD, UML use case, activity, sequence, and class diagrams) map directly to the implemented codebase.

14.1 References

1. Laravel Documentation. https://laravel.com/docs
2. Stripe Checkout Documentation. https://stripe.com/docs/payments/checkout
3. Laravel Reverb. https://laravel.com/docs/reverb
4. Somerville, I. Software Engineering. Pearson.
5. Elmasri, R. & Navathe, S. Fundamentals of Database Systems.

A. Demo Accounts and Seed Data

Password for all seeded accounts: password

Role	Email	Notes
Admin	admin@fixflow.test	Full system access
Customer	customer@fixflow.test	John Customer; seeded repairs/chat
Technician	technician@fixflow.test	Mike Torres (approved)
Technician	technician2@fixflow.test	Lisa Chen (approved)
Technician	technician3@fixflow.test	David Park (approved)
Applicant	applicant@fixflow.test	Pending admin approval

Payment Demo Repairs

- RR-DEMO-PAY: completed + draft invoice ($230) for admin send flow.
- RR-DEMO-PAY-2: completed + unpaid invoice ($150) for immediate Stripe pay.

B. Route Map and Repository

Method	Path	Purpose
GET	/	Marketing home
GET/POST	/login, /register	Auth
GET/POST	/forgot-password, /reset-password	Password reset
GET	/dashboard/*	Role dashboards
*	/repair-requests...	Repair lifecycle
GET	/messages, /messages/{rr}	Inbox
GET/POST	/repair-requests/{rr}/messages*	Chat API
*	/invoices...	Invoice + Stripe
GET	/reports	Admin analytics
GET/POST	/technician-applications...	Tech hiring
POST	/stripe/webhook	Stripe events
POST	/preferences/table-density	UI cookie

Repository: https://github.com/mehadi105/FixFLow
UI screenshots of the running demo are included in Appendix C.

C. User Interface Screenshots

Screenshots from the local demo (http://127.0.0.1:8000) with seeded accounts. Tall captures are cropped to the primary viewport so each figure fills the page width.

Public marketing hero: product pitch, CTAs, live dashboard preview, and portal entry points.

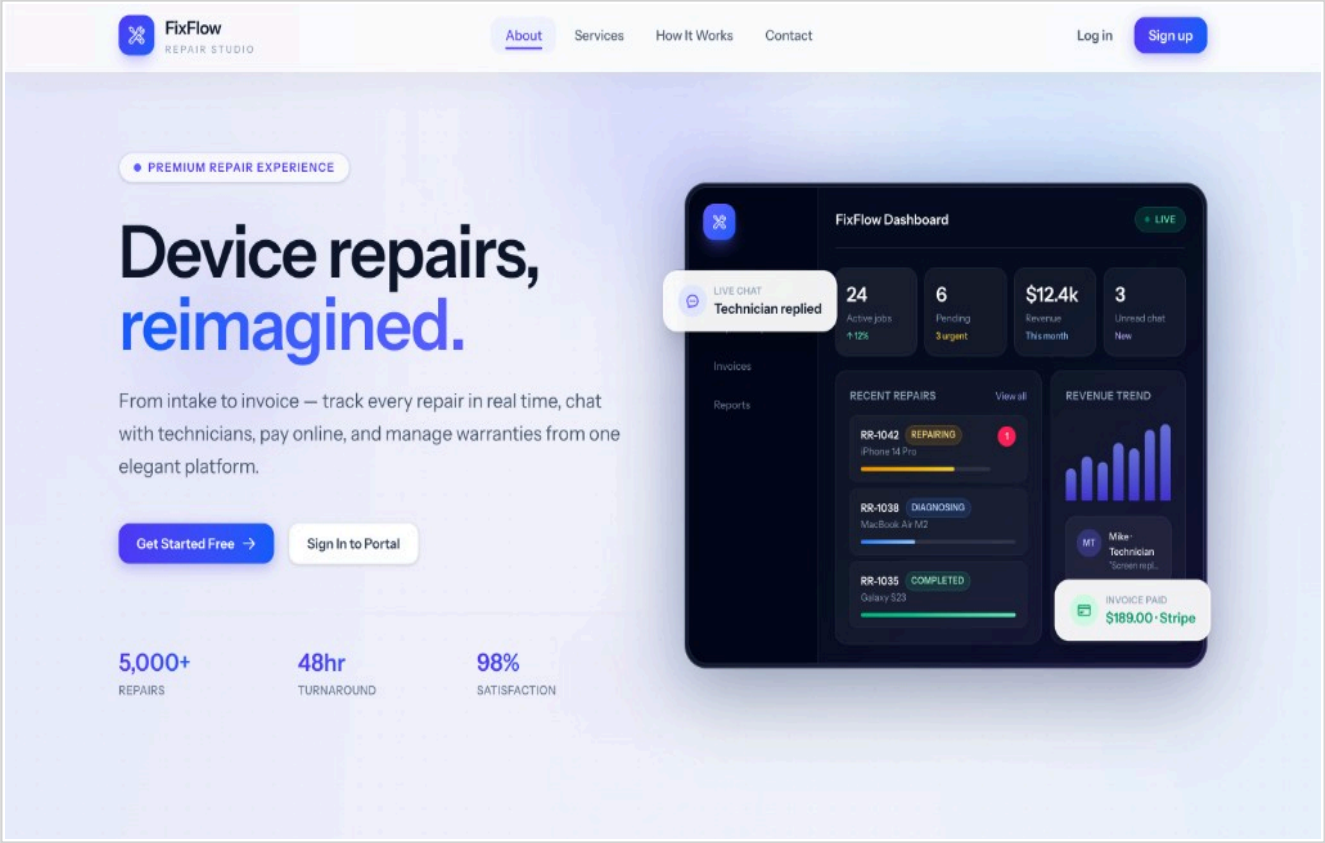

Figure C.1 - Marketing home hero (landing page).

Session login for all roles; register as customer or apply as technician.

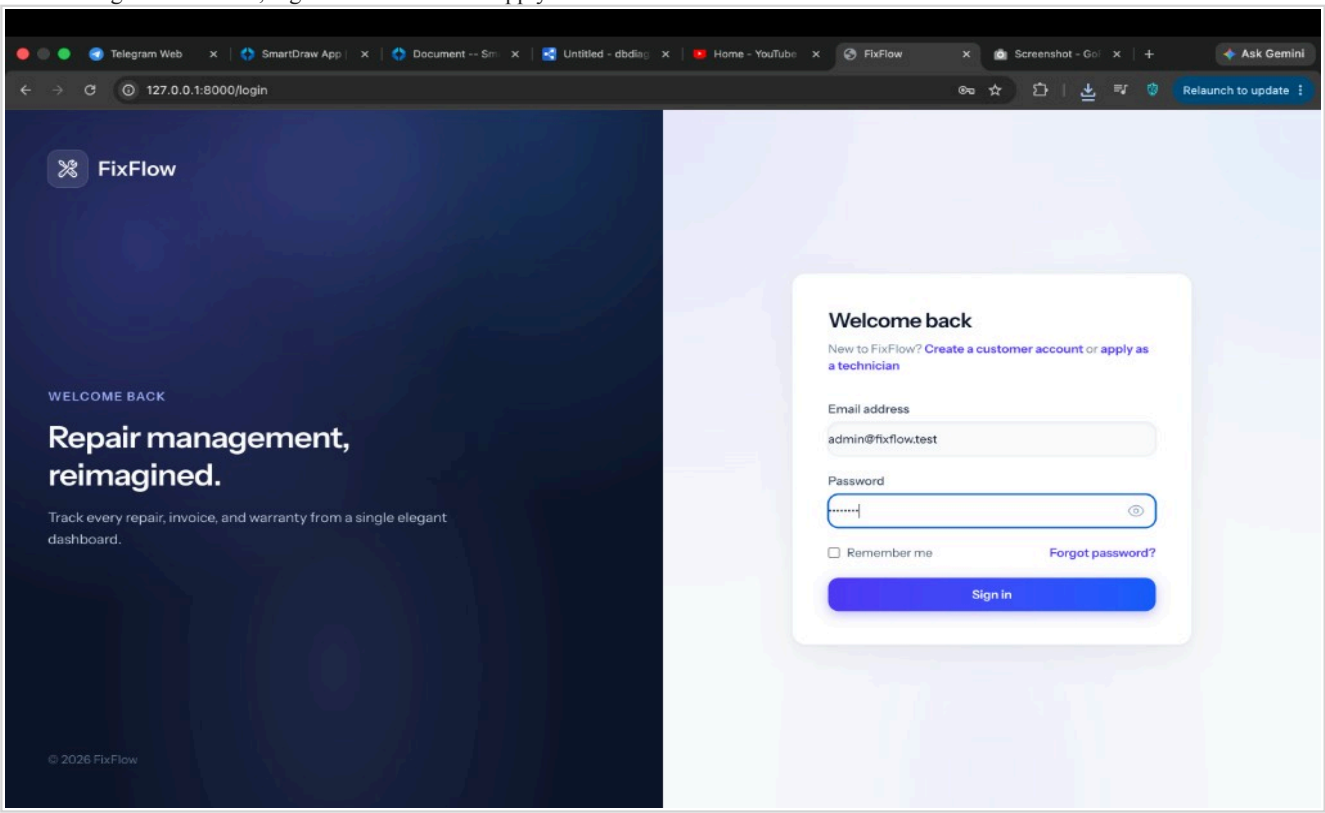

Figure C.2 - Sign-in screen.

KPIs, recent repairs, and status doughnut (includes Quoted / Declined).

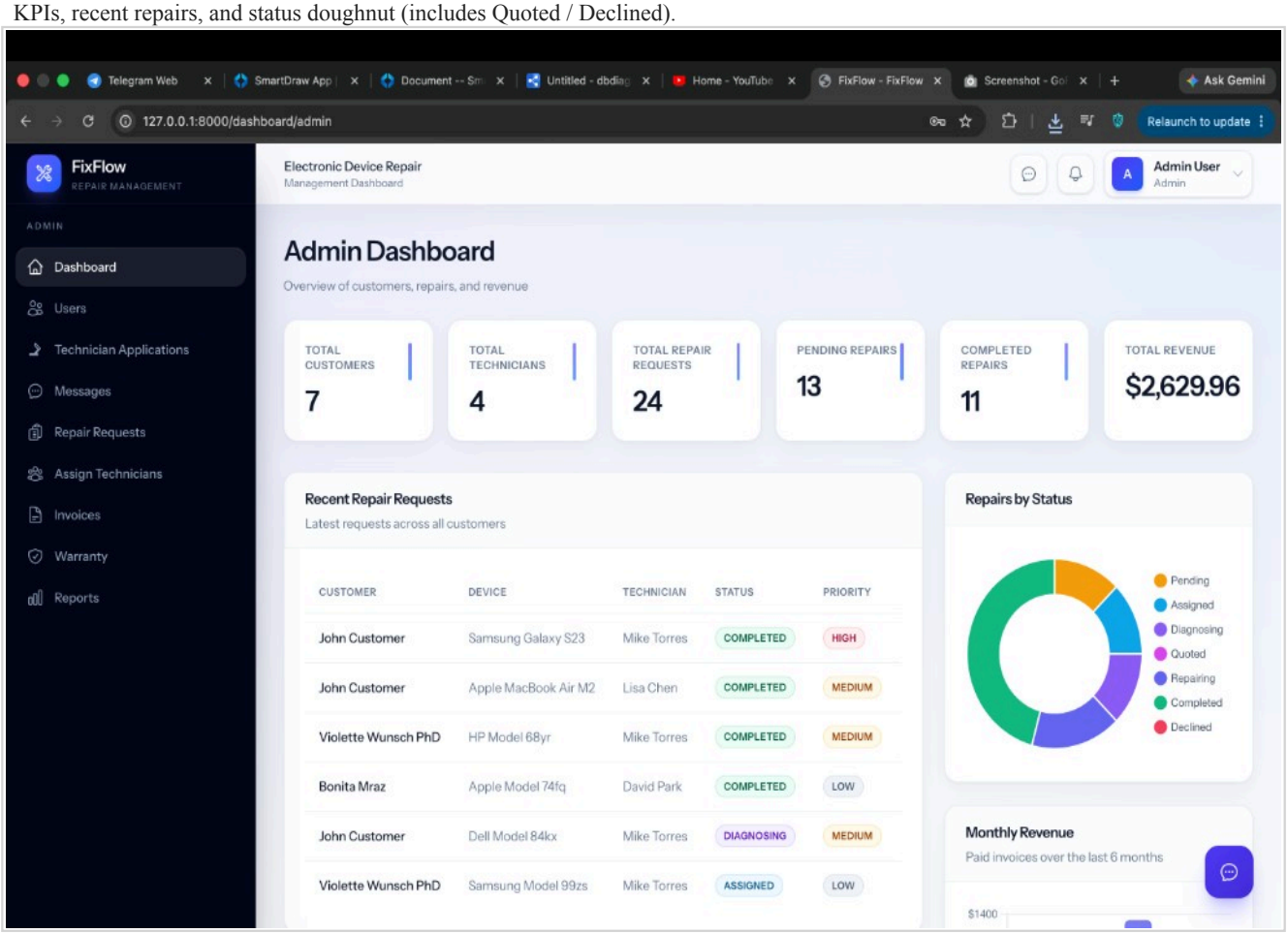

Figure C.3 - Admin dashboard.

Request totals, warranties, and recent repairs with View actions.

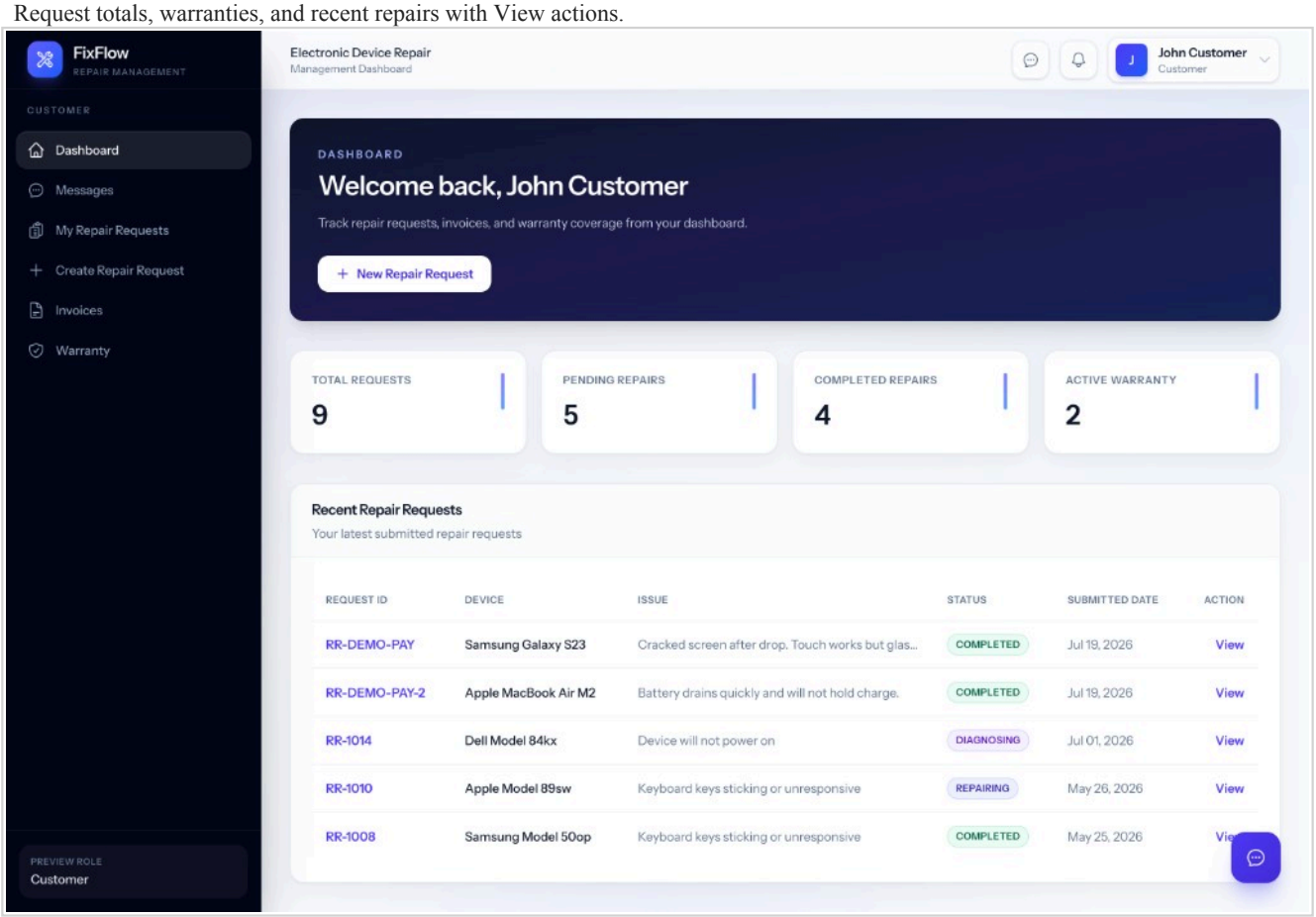

Figure C.4 - Customer dashboard.

Customer submits device details, priority, issue text, and optional image.

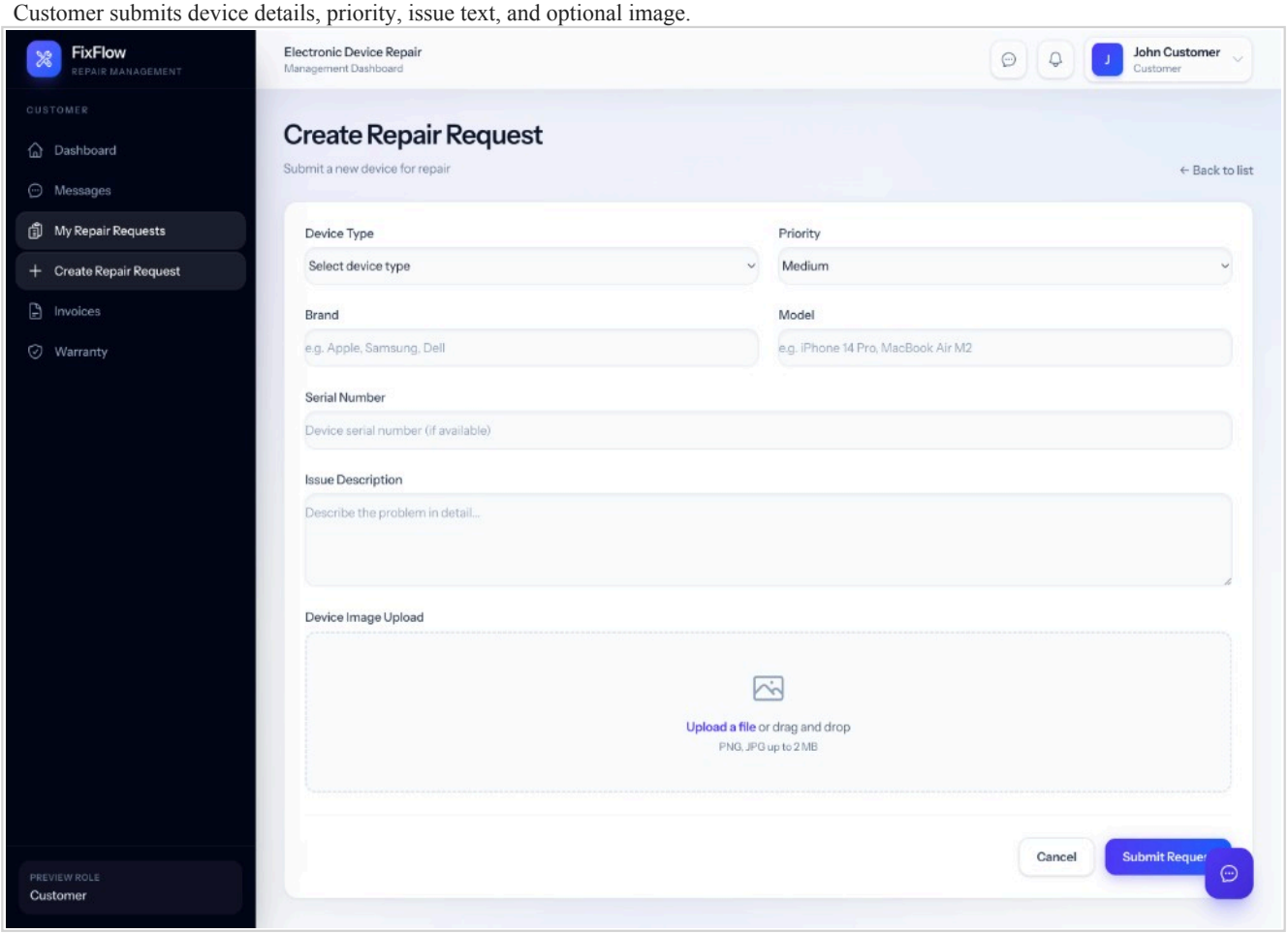

Figure C.5 - Create repair request.

Timeline, assignment, diagnosis/quote panel, invoice and warranty shortcuts.

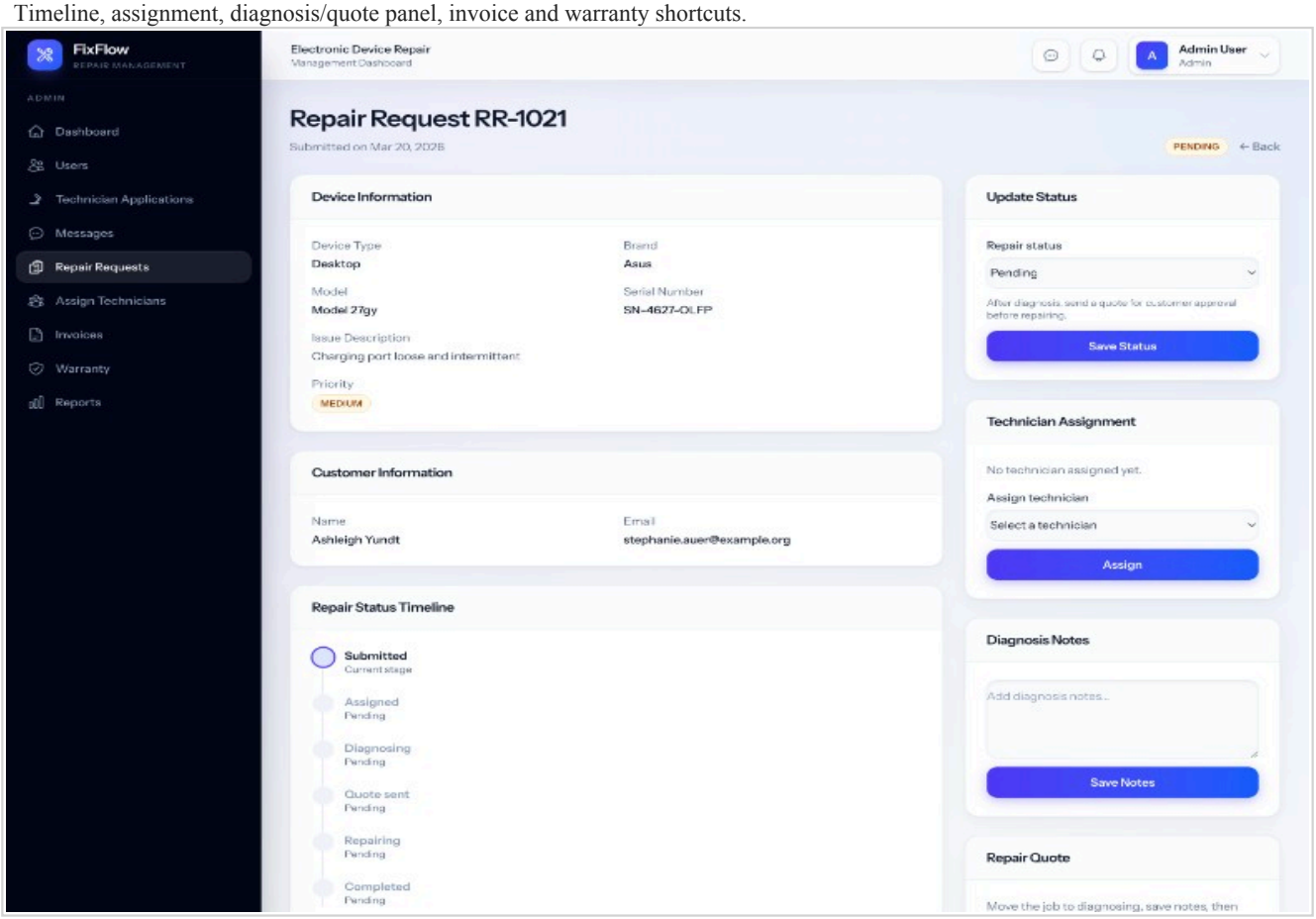

Figure C.6 - Repair request detail (admin).

One chat thread per repair; realtime updates via Laravel Reverb.

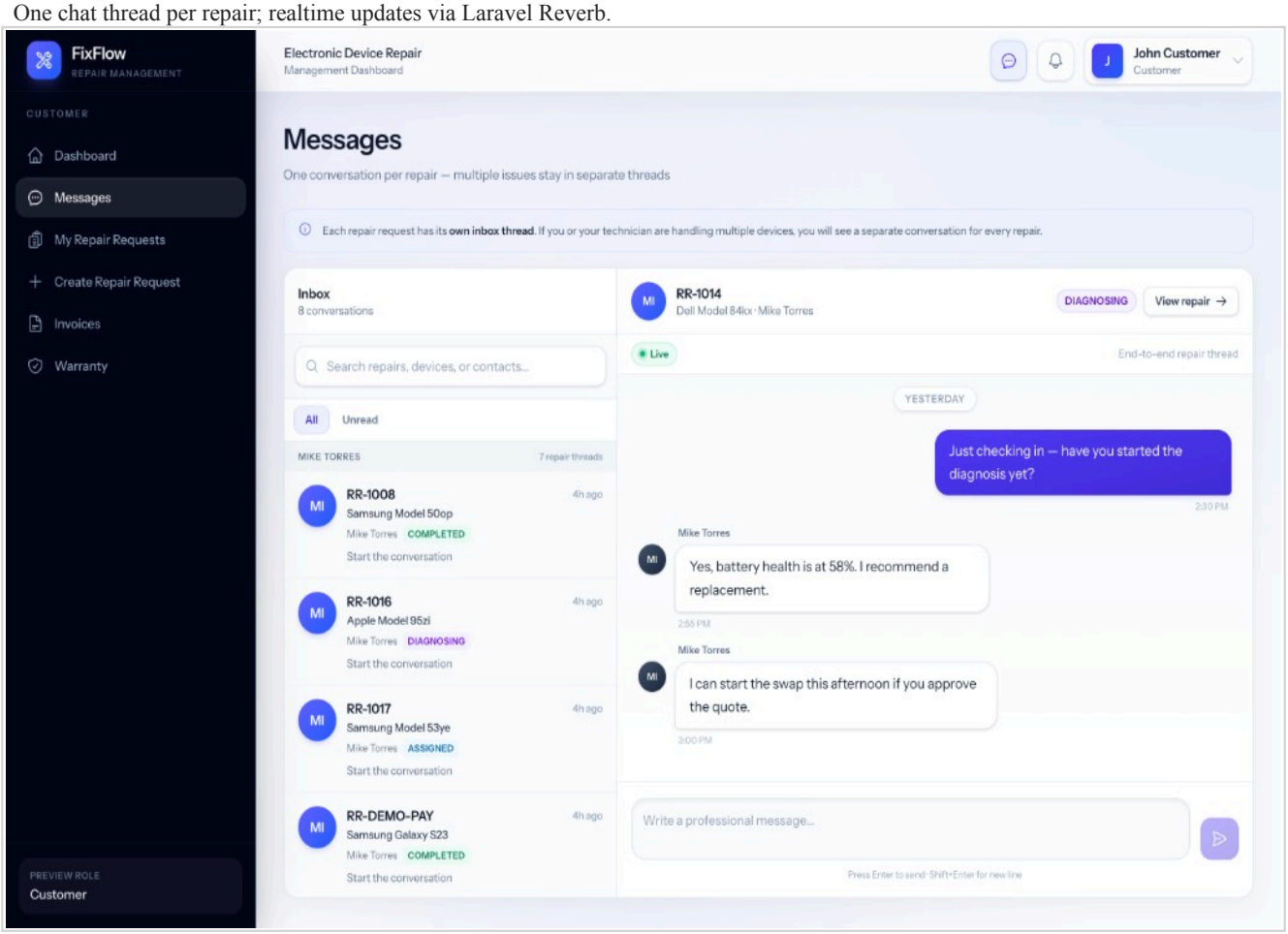

Figure C.7 - Live messages inbox.

Revenue, status mix, monthly volume, and technician completion rates.

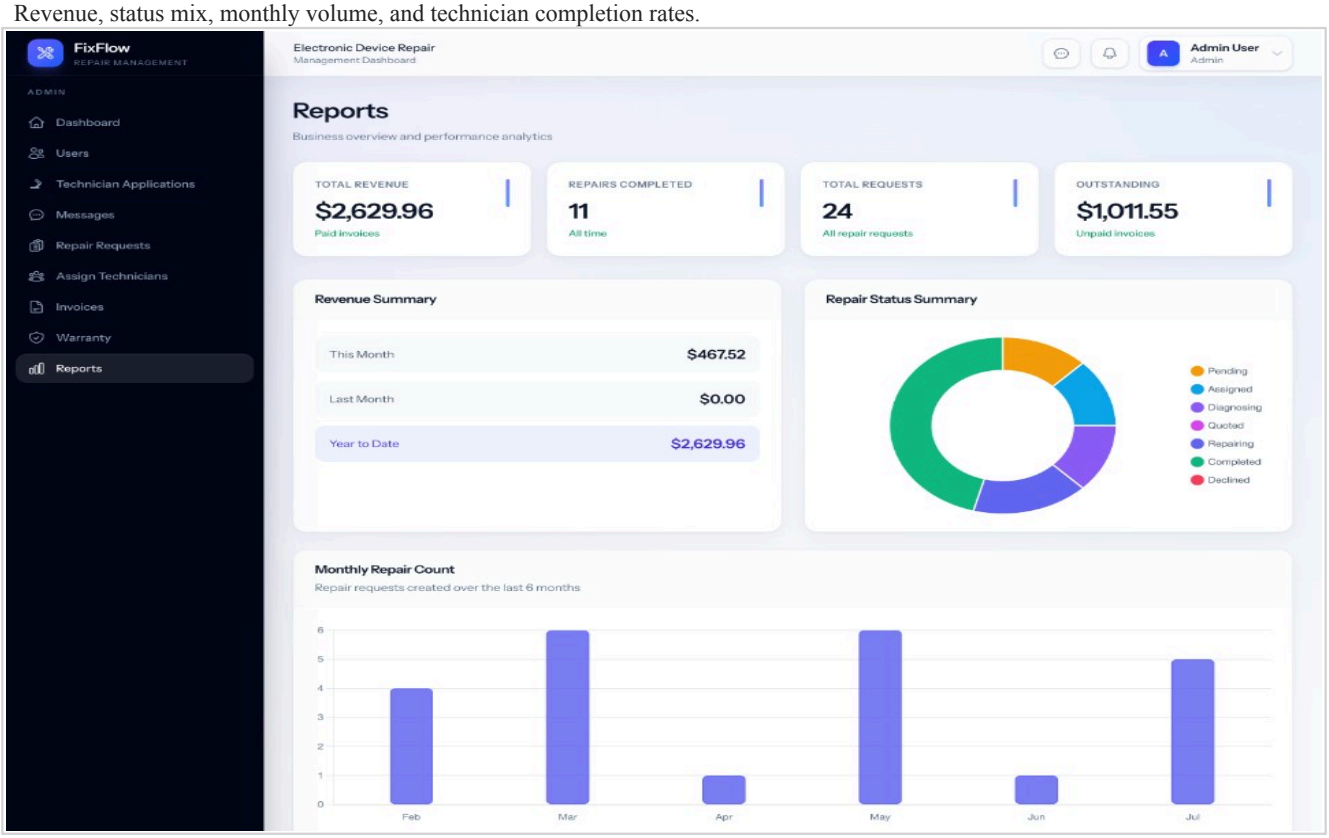

Figure C.8 - Admin reports.

End of Report — FixFlow Final Project Documentation.